

Scan and audit my entire codebase and lets prepare an implementation plan to wire, polish, clean up, and prepare the app for production using industry standard best code practices... There's a user-app(residents), guard app, and a superadmin dashboard.. this scan and audit must be deep and comprehensive in order to archive the full stack and production ready app...

Explored 3 files, 2 searches, 14 lists

Ran git status -s

Explored 29 files, 8 searches, 5 lists

Thanks for trusting me with this — I dug through the repo and there's a lot of solid groundwork here across the resident/guard/admin stack, but there are a few critical blockers to fix before production.

Audit Snapshot

- Stack: Expo (resident + guard), Next.js (superadmin), Express + Supabase (backend + DB).
- Auth + data access: currently split across direct Supabase client usage and backend routes, with multiple admin paths.

Critical Findings (must fix before any production use)

- supabase.ts: Service-role key is hardcoded and supabaseAdmin is imported in client hooks/pages, which would ship the key to browsers and expose full DB access. Action: remove from client, move privileged ops server-side, rotate the key immediately.
- options.ts: Auth accepts hardcoded passwords and fallback demo users, and hardcodes secret. Action: integrate Supabase auth (or your backend), remove demo paths, move secret to env.
- *.ts: Auth/permission middleware is commented out or missing for many write endpoints.
Examples: guards.ts, amenities.ts, payment.ts, maintenance.ts, complaint.ts, visitor.ts, message.ts. Action: enforce requireAuth/RBAC on all routes.
- upload.ts and upload.ts: Public upload endpoint, no size/type limits, returns public URLs. Action: add auth + validation + signed URLs.

High Priority Issues

- route.ts and other API routes: use supabaseAdmin without session checks, effectively unauthenticated admin endpoints.
- middleware.ts: exports both middleware and next-auth default, which can override or mis-apply auth middleware.
- next.config.mjs: ignoreBuildErrors and ignoreDuringBuilds allow production builds with TS/ESLint failures.
- test-direct-supabase.html and superadmin/src/app/test-*: test pages should not ship.
- Hardcoded Supabase anon keys and API base URLs in mobile configs: app.json, app.config.js, supabase.{js,ts}, app.json.
- supabase.ts: server client uses autoRefreshToken and persistSession, which are browser-specific and unnecessary on server.

Medium Priority Issues

- Open CORS with no origin restrictions: app.ts.
- No request validation across backend controllers; errors are passed directly to clients.
- auth.ts: logout is likely a no-op because server has no session to sign out.
- Multiple Supabase migration roots: supabase/migrations, backend/supabase/migrations, superadmin/supabase/migrations risk schema drift.
- Multiple lockfiles in superadmin (package-lock.json, yarn.lock, bun.lockb) cause nondeterministic builds.
- No CI workflows in .github for lint/test/build.

Low Priority Issues

- Lots of backup artifacts (*.bak, *.backup) in user/ and Guard/.
- node_modules/ present in repo directories.

Implementation Plan (Production Readiness)

1. Phase 1: Security hardening — rotate Supabase keys; remove service-role key from client; move admin data access behind server/API; enforce auth on all backend routes; lock down uploads with limits + signed URLs; remove demo auth/passwords; move NextAuth secret to env.
2. Phase 2: Auth & RBAC consolidation — define roles/permissions centrally; enforce RBAC in backend middleware; apply Supabase RLS policies; ensure mobile apps only use anon key with RLS.

3. Phase 3: API stabilization — add schema validation (Zod/Yup) to backend requests; normalize error responses; add pagination; add OpenAPI docs (your server log claims /api-docs but it isn't implemented in backend/src).
4. Phase 4: Configuration & build hygiene — remove hardcoded keys in mobile apps; pick one Expo config file (app.json or app.config.js) for user app; standardize package manager; remove build-ignoring flags; add CI for lint/test/build.
5. Phase 5: Data & migrations — select a single source of truth for Supabase migrations; generate database.types.ts from that source for all apps; align seeds and RLS policies.
6. Phase 6: Quality & observability — add logging/monitoring (Sentry/Logtail), rate limiting, and security headers; add backend unit tests + integration tests; add Playwright smoke tests for superadmin; add mobile regression tests.

Assumptions I Made

- Supabase is your primary database and auth provider.
- The Express backend should be the privileged API for admin operations and sensitive writes.
If either of those is wrong, I'll adjust the plan.

What I didn't do

- I didn't run tests or build any app in this pass.

If you want, I can start by fixing the critical items first. To proceed efficiently, which direction do you want?

1. Keep the Express backend as the single privileged API (recommended for admin + sensitive data).
2. Move privileged logic into Next.js API routes and reduce the Express backend.
3. Go mostly direct-to-Supabase with strict RLS + Edge Functions.

Once you pick, I can start implementing Phase 1 and we'll work through the rest together.

Lets fix everything in the implementation plan in sequence so we do not leave any stone unturned... and if this (The Express backend should be the privileged API for admin operations and sensitive writes.) follows the industry standard approach and best coding practices, then lets do it... PLEASE MAKE SURE YOU DO NOT CHANGE UI DESIGNS OR AYOUT OR MESS UP THE CODE BECAUSE WE ARE PREPARING FOR PRODUCTION... IF YOU DONT UNDERSTAND ANYTHING OR NEED TO MAKE MAJOR CHANGES THAT ARE

NOT PART OF THE IMPLEMENTATION PLAN, PLEASE ASK... NOW LETS GET TO WORK... GIVE ME YOUR BEST.

Ran ls superadmin/src/app/(other)/auth

Explored 3 files, 1 search, 2 lists

Ran sed -n '1,220p' superadmin/src/app/(admin)/settings/admin/roles/page.tsx

Explored 3 files, 1 list

Edited
guards.ts
+7
-7

Edited
amenities.ts
+7
-7

Edited
units_enhanced.ts
+3
-3

Edited
payment.ts
+9
-9

Edited
complaint.ts

+4

-4

Edited
maintenance.ts

+4

-4

Edited
visitor.ts

+7

-7

Edited
message.ts

+3

-3

Edited
notice.ts

+2

-2

Edited
society.ts

+2

-2

Edited
unit.ts

+2

-2

Edited
upload.ts

+28
-2

Edited
upload.ts
+5
-3

Edited
upload.ts
+5
-2

Edited
errorHandler.ts
+12
-2

Edited
app.ts
+28
-2

Edited
package.json
+1
-0

Edited
supabase.ts
+2
-2

Edited
options.ts
+31
-64

Explored 1 file, 1 search

Created
next-auth.d.ts
+18
-0

Edited
useSignIn.ts
+2
-2

Edited
supabase.ts
+3
-19

Created
supabaseAdmin.ts
+17
-0

Explored 2 files, 2 searches

```
Ran python - <<'PY' import os import re def should_process(path: str) -> bool: if
not path.endswith(('.ts', '.tsx', '.js', '.jsx')): return False if '/app/api/' in path: return
False if '/lib/' in path: return False return True root = 'superadmin/src'
replacements = 0 files = 0 for dirpath, _, filenames in os.walk(root): for name in
filenames: path = os.path.join(dirpath, name) if not should_process(path):
continue with open(path, 'r', encoding='utf-8') as f: content = f.read() orig = content
# Normalize import statements content = re.sub(r"import\s+\{\s*supabase\s*,
\s*supabaseAdmin\s*\}\s+from\s+[""]@/lib/supabase[""];?", "import { supabase }
from '@/lib/supabase';", content) content = re.sub(r"import\s+\
{\s*supabaseAdmin\s*\}\s+from\s+[""]@/lib/supabase[""];?", "import { supabase }
from '@/lib/supabase';", content) content = re.sub(r"import\s+\{\s*supabase\s*,
\s*supabaseAdmin\s*\}\s+from\s+[""]../lib/supabase[""];?", "import { supabase }
from '../lib/supabase';", content) content = re.sub(r"import\s+\
{\s*supabaseAdmin\s*\}\s+from\s+[""]../lib/supabase[""];?", "import { supabase }
from '../lib/supabase';", content) content = re.sub(r"import\s+\
{\s*supabaseAdmin\s*\}\s+from\s+[""]../lib/supabase[""];?", "import { supabase }
```

```
from '../lib/supabase';", content) # Replace identifier usage content =
content.replace('supabaseAdmin', 'supabase') if content != orig: with open(path,
'w', encoding='utf-8') as f: f.write(content) files += 1 replacements += 1
print(f'Updated {files} files") PY
```

```
Ran python - <<'PY' import os import re root = 'superadmin/src/app/api' for
dirpath, _, filenames in os.walk(root): for name in filenames: if not
name.endswith(('ts', 'tsx', 'js', 'jsx')): continue path = os.path.join(dirpath, name)
with open(path, 'r', encoding='utf-8') as f: content = f.read() new_content =
re.sub(r"from\s+['\"]@/lib/supabase[\"']", "from '@/lib/supabaseAdmin'", content) if
new_content != content: with open(path, 'w', encoding='utf-8') as f:
f.write(new_content) PY
```

Explored 6 files, 1 search

Edited
app.config.js
+3
-3

Edited
app.json
+2
-2

Edited
app.json
+2
-2

Created

app.config.js
+14
-0

Edited
supabase.js
+11
-4

Edited
supabase.ts
+11
-4

Edited
supabase.js
+12
-2

Edited
directSupabase.js
+12
-3

Edited
supabaseConfig.js
+12
-8

Edited
moduleSettingsService.js
+12
-2

Explored 1 file

Edited
moduleSettingsService.js

+12
-2

Explored 1 file

Deleted
test-direct-supabase.html
+0
-44

Explored 1 file, 2 searches

Edited
.env.example
+8
-4

Edited
app.json.with-agora
+2
-2

Explored 1 file

Edited
env-setup.js
+2
-2

Explored 1 file

Edited
testCommunityLogic.js
+6
-3

Explored 1 file

Edited

updateTypes.js
+6
-2

Explored 1 file

Edited
create-storage-bucket.js
+5
-7

Explored 1 file

Edited
directDatabaseFix.js
+5
-8

Explored 1 file

Edited
testRelationships.js
+6
-2

Explored 1 file

Edited
assignUnit.js
+6
-2

Explored 2 files, 4 searches

Edited
options.ts
+1
-1

Explored 1 file, 1 search

Created
.env.example
+5
-0

Created
.env.example
+3
-0

Created
.env.example
+5
-0

Edited
options.ts
+4
-0

Edited
errorHandler.ts
+5
-0

Edited
route.ts
+16
-0

Explored 1 file

Edited
route.ts
+12
-0

Explored 1 file

Edited
route.ts
+12
-0

Explored 1 search

Edited
useSignUp.ts
+22
-44

Explored 1 file, 1 search

Edited
app.config.js
+1
-1

Explored 2 files, 2 searches

Edited
seedMemberData.js
+6
-3

Ran git status -s

Explored 1 search

Ran sed -n '400,440p' superadmin/src/app/(admin)/settings/guards/page.tsx

Explored 1 file

Edited
page.tsx
+0
-2

Explored 1 file, 1 search

Edited
guards.ts
+3
-3

Edited
amenities.ts
+3
-3

Edited
units_enhanced.ts
+3
-3

Edited
payment.ts
+1
-1

Explored 1 file

Edited
payment.ts
+1
-1

Awesome — thanks for the clear direction; this is a strong, multi-app setup and I'm excited to get it production-ready with you.

Phase 1 Done (Security Hardening)

- Enforced auth on backend routes and fixed routing edge cases: *.ts (guards/amenities/units/payments/etc. now require auth/admin; reordered search/stats routes to avoid shadowing).
- Secured uploads (auth + size/type limits + random filenames) and improved error handling: upload.ts, upload.ts, upload.ts, errorHandler.ts.
- Added baseline security headers + request size limits + CORS allowlist: app.ts (added helmet, CORS_ORIGINS, JSON limit).
- Removed hardcoded Supabase keys from client code and configs:
 - Superadmin: supabase.ts now uses NEXT_PUBLIC_* env only; created server-only supabaseAdmin.ts.
 - Mobile: app.json, app.json, app.json.with-agora now blanked; new app.config.js; app.config.js uses env + fallback.
 - User/Guard Supabase clients now pull from Expo extra env: supabase.
{js,ts}, supabase.js, directSupabase.js, supabaseConfig.js, moduleSettingsService.js, moduleSettingsService.js.
- Locked NextAuth down (no demo users/passwords, real Supabase auth + admin role check):
 - options.ts
 - Added session typing: next-auth.d.ts
 - Removed demo defaults: useSignIn.ts
- Removed plaintext password writes from admin sign-up flow: useSignUp.ts
- Protected superadmin API routes with admin session checks:
route.ts,
route.ts,
route.ts
- Deleted insecure test asset: test-direct-supabase.html
- Added .env.example files: .env.example, .env.example, .env.example, and sanitized .env.example.

Important Notes / Required Actions

1. Rotate Supabase keys now. They were previously committed in multiple files. Generate new keys in Supabase and update your .env / deployment secrets.
2. Install backend deps (new helmet):
Run npm install in backend/.
3. Set required envs (minimum):
 - Superadmin: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, NEXTAUTH_SECRET, NEXTAUTH_URL
 - Backend: SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, CORS_ORIGINS, UPLOAD_*
 - Mobile: EXPO_PUBLIC_SUPABASE_URL, EXPO_PUBLIC_SUPABASE_ANON_KEY, API_BASE_URL
4. Potential behavior change: superadmin now uses the anon key on the client (no service-role key in browser). Admin screens will only work if RLS allows admin access. In Phase 2 we'll either:
 - Move admin data access behind the Express API (your preferred standard), or
 - Update RLS policies to allow admin roles safely.

I did not run tests/builds yet.

Question (needed before Phase 2):

Do you want the superadmin sign-up page to be:

1. Disabled by default (recommended for production),
2. Invite-only, or
3. Open self-sign-up (not recommended)?

Once you answer, I'll move into Phase 2: RBAC consolidation and migrating privileged admin data access to the Express backend without touching any UI layout.

53 files changed

+447

-311

Undo

Guard/.env.example

Guard/app.config.js

Guard/app.json

Guard/services/moduleSettingsService.js

backend/.env.example

backend/package.json

backend/src/app.ts

backend/src/controllers/upload.ts

backend/src/lib/supabase.ts

backend/src/middleware/errorHandler.ts

backend/src/routes/amenities.ts

backend/src/routes/complaint.ts

backend/src/routes/guards.ts

backend/src/routes/maintenance.ts

backend/src/routes/message.ts

backend/src/routes/notice.ts

backend/src/routes/payment.ts

backend/src/routes/society.ts

backend/src/routes/unit.ts

backend/src/routes/units_enhanced.ts

backend/src/routes/upload.ts

backend/src/routes/visitor.ts

backend/src/services/upload.ts

superadmin/.env.example

superadmin/public/test-direct-supabase.html

superadmin/src/app/(admin)/settings/guards/page.tsx

superadmin/src/app/(other)/auth/sign-in/components/useSignIn.ts

superadmin/src/app/(other)/auth/sign-up/components/useSignUp.ts

superadmin/src/app/api/auth/[...nextauth]/options.ts

superadmin/src/app/api/debug-user-profiles/route.ts

superadmin/src/app/api/module-settings/route.ts

superadmin/src/app/api/test-user-profiles/route.ts

superadmin/src/lib/supabase.ts

superadmin/src/lib/supabaseAdmin.ts

superadmin/src/types/next-auth.d.ts

user/.env.example

user/app.config.js

user/app.json

user/app.json.with-agera

user/env-setup.js

user/lib/supabase.js

user/scripts/assignUnit.js

user/scripts/create-storage-bucket.js

user/scripts/directDatabaseFix.js

user/scripts/seedMemberData.js

user/scripts/testCommunityLogic.js

user/scripts/testRelationships.js

user/scripts/updateTypes.js

user/services/moduleSettingsService.js

user/utls/directSupabase.js

user/utls/supabase.js

user/utls/supabase.ts

user/utls/supabaseConfig.js

I will go with this one..... Disabled by default (recommended for production).... Remember that the superadmin panel will be scoped for admins, like agency and facility managers so we need to use the industry standard approach in this regard... before you go ahead, explain and tell me which is the best to go for so it will make it easy for production...

Totally hear you, and your choice makes sense — for production, disabling self-signup on the superadmin panel is the safest and most standard approach.

Best-practice approach for production (recommended)

For an admin/superadmin panel that's scoped to agency + facility managers, the most industry-standard pattern is:

1. Central Auth Provider
Use Supabase Auth for login (or an external IdP later if you want SSO like Okta/Google Workspace).
The admin panel never stores passwords itself.
2. Backend as the Privileged API (your preferred standard)
The Express backend becomes the only place that performs privileged reads/writes.
It validates the Supabase JWT, checks roles/permissions, and then uses the service-role key server-side.
3. RBAC + Tenant Scoping
Users are assigned roles like superadmin, agency_manager, facility_manager,

etc.

The backend enforces:

- role permissions
- tenant scoping (agency/facility/community)
- least privilege (only what they need)

4. Frontends are always least-privilege

- Mobile apps only use the anon key + RLS.
- Superadmin UI uses anon key for auth only, and calls the backend for any privileged data.

This is the most production-ready and scalable way to keep admin access safe, audit-able, and easy to extend.

About disabling signup

We can disable sign-up without changing the UI layout by:

- keeping the page visible but making the signup request fail with a clean “Disabled in production” message, or
- hiding the sign-up link (this is a UI change, so only if you want it).

Before I proceed, I need two small clarifications:

1. Which admin roles should exist now?

I recommend at minimum:

- superadmin
- agency_manager
- facility_manager
- admin (community-level)
Please confirm or adjust.

2. Tenant scope rules:

Should agency_manager see multiple facilities?

Should facility_manager only see one facility/community?

Once you confirm those, I'll:

- Disable sign-up in production (server-side, no UI design change)
- Move all privileged superadmin data access to the Express backend
- Enforce RBAC there

- Lock down any remaining admin-only routes

If that plan aligns, say “Proceed” and I’ll move forward.

I will go with these as recommended.... superadmin

agency_manager

facility_manager

admin (community-level)..... BUT before we begin, I need to know, what will be the industry standard way or giving access to these roles? do i open the sign-up for them later when we are done or do I use invite? Lets do a deep research and find out the best approach so we get everything right before we move forward....

Searched web for OWASP ASVS requirements for privileged accounts and account management invite or admin approval

Great call on pausing here—this access-governance decision is one of those “get it right once, benefit forever” moments.

Industry-standard answer (short version)

For admin/superadmin panels, open self-signup is not considered best practice.

The production-grade pattern is invite-only or IdP-managed provisioning (SSO/ SCIM/JIT), with MFA required for all admin roles, centralized access control, and least-privilege roles. (blog.netwrix.com)

Why invite-only (and/or SSO) is standard

1. Privileged accounts require stronger controls: industry guidance calls for MFA on admin access and centralized access control, not open registration. (blog.netwrix.com)
2. Authentication assurance matters: NIST 800-63B defines assurance levels and stronger authenticators for sensitive access, which aligns with “no public admin signup.” (csrc.nist.gov)
3. Modern enterprise practice is JIT + delegated admin: Just-in-time provisioning and delegated administration are typical in SaaS to reduce manual errors and keep access scoped to tenant roles. (loginradius.com)

Best approach for your setup (recommended)

1. Now (production-ready baseline): Invite-only + MFA
 - No public sign-up for admin roles.
 - Superadmin creates agency_manager, facility_manager, admin users.
 - Admin roles are enforced by backend + permissions.
 - Require MFA for all admin roles.

2. Next (enterprise-ready): SSO + SCIM/JIT

- Integrate SSO (SAML/OIDC) for organizations.
- Enable JIT provisioning or SCIM for lifecycle automation.
- Support delegated admin per tenant (agency/facility). (loginradius.com)

So the best “production-easy” path is:

Invite-only now, with a planned SSO/JIT path later. This is the common, least-risk pattern for admin panels handling sensitive data. (blog.netwrix.com)

Before I implement Phase 2, please choose your invite workflow:

1. Backend-only provisioning (fastest): superadmin creates accounts via backend API, and you distribute credentials manually (or we add email later).
2. Invite email flow: backend generates a token and sends an invite email link (more work but clean UX).

And one more confirmation:

- Should I hard-block the sign-up page (returns 404/redirect) or keep it visible but disabled (shows “invite-only” message)? This won’t change layout, just behavior.

Once you pick those two, I’ll move into Phase 2: RBAC consolidation and moving admin data access fully behind the Express backend.

Lets implement these two if only they follow best coding practices so the superadmin becomes dynamic, giving it mutliple ways of onboarding new admins.... Backend-only provisioning (fastest): superadmin creates accounts via backend API, and you distribute credentials manually (or we add email later). Invite email flow: backend generates a token and sends an invite email link (more work but clean UX)..... WE WILL ALSO keep the sign up form visible but disabled (shows “invite-only” message)? This won’t change layout, just behavior.

Explored 1 search, 1 list

```
Ran python - <<'PY' import json, re from pathlib import Path path =  
Path('backend/src/lib/database.types.ts') text = path.read_text() # find profiles  
table definition in generated types m = re.search(r"profiles:\s*\{[\s\S]*?Row:\s*\s*\}  
{([\s\S]*?)\}\s*Insert", text) if m: fields = m.group(1).strip().split('\n')  
print('\n'.join(fields[:60])) else: print('not found') PY
```

Explored 3 files, 1 search, 1 list

```
Ran grep -R --line-number --exclude-dir=node_modules --exclude-dir=.git "create
user" -n superadmin/src/app/(admin)/settings/users superadmin/src/hooks/
useUserProfiles.ts
```

Explored 3 files, 2 searches, 2 lists

```
Edited
auth.ts
+2
-2
```